

# Aprendizado por reforço profundo para locomoção bípede de um robô humanoide em simulação MuJoCo

**Letizia L. Baptistella, Manuella F. Peres, Rafaela A. de Campos**

*Ciência da Computação – Centro Universitário FEI*

*uniflbaptistella@fei.edu.br, unifmperes@fei.edu.br, unifrcampos@fei.edu.br*

**Orientador, Prof. Dr. Isaac Jesus da Silva**

*Departamento de Ciência da Computação – Centro Universitário FEI*

*isaacjesus@fei.edu.br*

**Coorientador, Reinaldo A. C. Bianchi**

*Departamento de Ciência da Computação – Centro Universitário FEI*

*rbianchi@fei.edu.br*

**Palavras-chave,** Deep Reinforcement Learning, Robótica Humanoide, Futebol de Robôs, MuJoCo, Aprendizado por Reforço

## I. Introdução

A locomoção bípede é um dos problemas mais desafiadores da robótica humanoide, pois exige equilíbrio dinâmico contínuo sobre uma base de apoio reduzida. De acordo com Kober et al. [15], diferentemente de robôs com rodas ou esteiras, humanoides passam parte do ciclo de movimento apoiados em apenas um pé, o que aumenta significativamente a instabilidade. Isso implica que pequenas perturbações, como irregularidades no solo ou contatos inesperados, podem resultar em quedas. Dessa forma, desenvolver controladores robustos para locomoção não é apenas um desafio técnico, mas um requisito fundamental para aplicações práticas como o futebol de robôs.

A qualidade da locomoção é o fator mais determinante para o desempenho competitivo no futebol de robôs, superando em importância qualquer estratégia de jogo. MacAlpine e Stone [18] analisaram competições da RoboCup 3D e mostraram que a velocidade e a confiabilidade da caminhada são os fatores que mais separam equipes vencedoras das demais. Isso acontece porque um robô que cai com frequência perde posicionamento em campo, gasta tempo em rotinas de recuperação e abre espaço para o adversário marcar. Esse resultado reforça que garantir uma locomoção rápida e robusta não é apenas uma questão técnica, mas sim um pré-requisito para que qualquer outra estratégia de jogo tenha chance de funcionar.

A abordagem baseada em Zero Moment Point (ZMP) apresenta limitações relevantes em ambientes dinâmicos como o futebol de robôs. Segundo Vukobratović e Borovac [32], o método depende de modelos matemáticos precisos para manter o equilíbrio, funcionando bem apenas em condições controladas. No entanto, como apontado por Maximo e Ribeiro [19], pequenas mudanças no hardware exigem recalibração manual frequente, além de dificultar a adaptação a perturbações não previstas. Isso torna o ZMP pouco flexível para cenários imprevisíveis, reforçando a necessidade de abordagens mais adaptativas, como o aprendizado por reforço.

O custo de manutenção do ZMP recai de forma desigual sobre equipes universitárias, que precisam dividir o tempo de desenvolvimento com outras frentes do projeto. A diferença está no custo relativo desse esforço, equipes com mais estrutura de pesquisa conseguem manter times dedicados à recalibração e ao ajuste fino do modelo, enquanto equipes como a RoboFEI precisam dividir esse tempo com outras frentes do projeto. Na prática, a cada semestre, boa parte do tempo disponível é consumida por ajustes no controlador causados por modificações no hardware, em vez de ser direcionada para o desenvolvimento de novas capacidades. Esse cenário reforça a necessidade de uma abordagem que, uma vez treinada, não precise de recalibração manual a cada mudança no sistema.

O Aprendizado por Reforço (RL) surge como uma alternativa adequada para lidar com essas limitações, pois permite que o agente aprenda diretamente a partir da interação com o ambiente. Conforme descrito por Sutton e Barto [30], o agente ajusta seu comportamento com base em recompensas, sem depender de um modelo explícito do sistema. Isso possibilita capturar automaticamente características difíceis de modelar, como dinâmicas não lineares e variações nos atuadores. Assim, o RL reduz a necessidade de ajustes manuais frequentes e se mostra mais adequado para ambientes dinâmicos como o futebol de robôs.

Este trabalho propõe usar DRL, especificamente o algoritmo PPO [27], para desenvolver uma política de controle que permita ao robô Atom da RoboFEI caminhar de forma mais estável e eficiente. O treinamento e a avaliação são feitos no simulador MuJoCo [31] integrado ao ambiente oficial da RoboCup 3D, garantindo que o agente aprenda diretamente nas condições em que vai competir. O trabalho se insere no contexto da RoboCup, competição internacional criada em 1997 com a meta de que, até meados deste século, um time de humanoides autônomos seja capaz de vencer a seleção humana campeã do mundo [14], e representa um passo concreto em direção a essa meta para a equipe.

## II. Revisão Bibliográfica

A revisão bibliográfica foi organizada em três tópicos, os fundamentos teóricos do RL e do DRL, os algoritmos disponíveis para controle contínuo e as aplicações específicas em locomoção humanoide e futebol de robôs. Cada conjunto de trabalhos influenciou diretamente escolhas do projeto, e o objetivo desta seção é mostrar como a literatura sustenta a viabilidade e a relevância do trabalho, além de justificar as decisões tomadas ao longo do caminho.

### A. Fundamentos de RL e DRL

O fundamento central do RL é o Processo de Decisão de Markov (MDP), que define de forma precisa como um agente aprende a partir da interação com o ambiente. Em um MDP, apresentado por Sutton e Barto [30], o agente observa o estado  $s$  do ambiente, escolhe uma ação  $a$ , recebe uma recompensa  $r$  e vai para um novo estado  $s'$ , com o objetivo de aprender uma política  $\pi$  que maximize a soma acumulada de recompensas ao longo do tempo. Para o Atom, o estado é o vetor de leituras dos sensores, a ação é o conjunto de torques aplicados nas articulações, e a recompensa incentiva o avanço sem quedas. Compreender o formalismo MDP foi importante porque ele deixa claro por que a função de recompensa tem tanto impacto no resultado final, assunto que retomará na Seção III.

A função valor-ação  $Q^\pi(s, a)$  é a ferramenta matemática que permite ao agente avaliar o quanto cada decisão contribui para o retorno futuro. Ela quantifica o retorno esperado ao executar a ação  $a$  no estado  $s$  e seguir a política  $\pi$  a partir daí [30], [33],

$$Q^\pi(s, a) = E_\pi \left[ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s, a_0 = a \right] \quad (1)$$

onde  $\gamma \in [0, 1]$  é o fator de desconto que controla o quanto as recompensas futuras influenciam as decisões presentes. Para locomoção, em que comportamentos desejados como manter o equilíbrio se manifestam ao longo de dezenas de passos, valores altos de  $\gamma$  são essenciais, pois garantem que o agente seja incentivado a sustentar posturas estáveis ao longo de uma passada completa em vez de otimizar apenas o instante atual. A escolha do valor exato de  $\gamma$  será definida durante os experimentos iniciais com o Atom e influencia diretamente o horizonte de planejamento do agente.

A equação de Bellman estabelece a relação recursiva que torna o aprendizado iterativo possível, permitindo que o agente se atualize passo a passo sem esperar o fim do episódio [30],

$$Q^\pi(s, a) = E \left[ r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s', a') \right] \quad (2)$$

Essa relação é poderosa mas possui um problema, quando a estimativa do valor do próximo estado é

imprecisa nas fases iniciais do treinamento, as atualizações ficam instáveis e o agente pode desaprender comportamentos que levou horas para adquirir, fenômeno conhecido como *catastrophic forgetting* [21]. Em locomoção bípede, isso é perigoso porque o agente pode investir muitas interações aprendendo a se manter de pé e perder esse comportamento após uma sequência de atualizações baseadas em episódios em que o robô já havia caído. Esse risco influenciou diretamente a escolha do tamanho dos lotes de atualização e da frequência de avaliação durante o treinamento do Atom.

Vale mencionar também a distinção entre políticas estocásticas e determinísticas, que é relevante para entender o comportamento do PPO em locomoção. Em políticas estocásticas, o agente não escolhe sempre a mesma ação para um dado estado, mas amostra de uma distribuição de probabilidade sobre as ações, o que garante exploração natural do espaço de estados durante o treinamento sem necessidade de adicionar ruído artificial. Para locomoção, essa característica é extremamente útil nas fases iniciais, quando o agente precisa experimentar diferentes padrões de movimento antes de convergir para um ciclo de passada estável. Na fase de avaliação, a política pode ser tornada determinística selecionando sempre a ação de maior probabilidade, eliminando variabilidade desnecessária sem nenhuma mudança na arquitetura da rede, o que será adotado nas avaliações semanais do Atom.

O objetivo de apresentar o DQN aqui é mostrar como ele estabeleceu as bases do DRL moderno ao demonstrar que redes neurais profundas conseguem aproximar funções de valor em espaços de estado complexos, pavimentando o caminho para os algoritmos de controle contínuo utilizados neste trabalho. O DQN (*Deep Q-Network*), proposto por Mnih et al. [21], combina o Q-Learning clássico [33] com redes neurais profundas, introduzindo duas inovações fundamentais para estabilizar o treinamento, uma memória de experiências (*replay buffer*) que armazena transições passadas e permite reamostrar experiências de forma descorrelacionada, e uma rede-alvo separada que mantém valores de referência estáveis por vários passos de atualização, evitando que os alvos mudem tão rapidamente quanto os pesos da rede principal. Essas contribuições foram decisivas porque, antes do DQN, tentativas de combinar Q-Learning com redes neurais profundas resultavam em treinamentos instáveis que divergiam rapidamente. O DQN foi o primeiro a mostrar, de forma sistemática, que redes profundas conseguem aprender representações úteis de estados de alta dimensionalidade sem engenharia manual de *features*, no experimento seminal, o agente aprendeu a jogar dezenas de jogos de Atari acima do nível humano usando apenas os pixels brutos da tela como entrada, sem qualquer conhecimento prévio das regras. No contexto deste projeto, esse resultado é relevante porque valida o uso de DRL para lidar com o vetor de alta dimensão

que descreve o estado do Atom, composto por ângulos, velocidades e sinais de contato das articulações. A limitação crítica do DQN, no entanto, é que ele só funciona com espaços de ações discretos, já que a maximização de  $Q$  sobre todas as ações possíveis se torna intratável quando o espaço de ações é contínuo, como é o caso do controle por torque das articulações do Atom. Riedmiller [25] já havia apontado que redes neurais conseguem aproximar a função de valor de forma eficiente mesmo quando o espaço de estados é grande, mas o problema do espaço de ações contínuo exigiu o desenvolvimento de algoritmos específicos, como o PPO, que será discutido na subseção seguinte. LeCun et al. [16] também explicam por que representações hierárquicas aprendidas automaticamente permitem generalizar diferentes variações de entrada que controladores clássicos normalmente só conseguem lidar com bastante ajuste manual, o que reforça a viabilidade de usar DRL para o Atom.

### B. Algoritmos para Controle Contínuo

O controle de articulações por torque exige um algoritmo capaz de lidar com espaço de ações contínuo, o que elimina diretamente o DQN como opção. O DQN maximiza  $Q$  sobre todas as ações possíveis, o que se torna inviável quando o espaço de ações é infinito, como é o caso do controle por torque das 20 articulações do Atom. O PPO foi escolhido como algoritmo principal por apresentar um bom equilíbrio entre estabilidade e desempenho em tarefas de controle contínuo. Schulman et al. [27] demonstram que o uso de uma função objetivo com *clipping* limita atualizações bruscas na política, reduzindo instabilidades durante o treinamento. Isso é particularmente importante em locomoção bípede, onde pequenas variações podem levar a quedas frequentes. Dessa forma, o PPO se mostra uma escolha adequada para o controle do robô Atom dentro das restrições do projeto.

$$L^{\text{CLIP}}(\theta) = E_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3)$$

onde  $r_t(\theta)$  é a razão entre as probabilidades da ação sob a nova e a antiga política, e  $\hat{A}_t = Q^\pi(s_t, a_t) - V^\pi(s_t)$  é a vantagem, que indica se a ação tomada foi melhor ou pior do que o esperado para aquele estado [30]. Quando  $r_t(\theta)$  sai do intervalo  $[1 - \epsilon, 1 + \epsilon]$ , o gradiente é cortado e a atualização é descartada nessa direção, impedindo atualizações desestabilizadoras sem o custo de resolver um problema de otimização restrita a cada passo como ocorre no TRPO [26]. Isso torna o PPO mais simples de implementar e ajustar, o que é uma vantagem relevante dentro do cronograma disponível.

Os demais algoritmos para controle contínuo foram avaliados e descartados por razões específicas que reforçam a escolha do PPO. O SAC [11] adiciona um termo de entropia ao objetivo para estimular a exploração, mas tem mais hiperparâmetros para calibrar

e exige ajustes de coeficiente de temperatura da entropia, que é sensível e pode comprometer a estabilidade exatamente quando o agente ainda está aprendendo comportamentos básicos como não cair. O TD3 [6] usa dois críticos independentes para reduzir a superestimação do valor de ação, mas sua documentação em locomoção bípede é bem menos extensa do que a do PPO, e o ajuste fino demandaria mais tempo do que possui-se disponível. O DDPG [17], que veio posteriormente ao TD3, foi descartado pelos mesmos motivos e pela instabilidade de treinamento conhecida em dinâmicas complexas como a de um humanoide. Essa avaliação comparativa deixou claro que o PPO é o ponto de partida mais adequado para o Atom, e os demais algoritmos ficam reservados para extensões futuras.

A escolha do PPO se dá por evidências sistemáticas na literatura de controle motor simulado. Duan et al. **duan2016** compararam diversos algoritmos em tarefas de controle motor e confirmaram que o PPO apresenta bom custo-benefício entre estabilidade e desempenho em sistemas com muitos graus de liberdade, ou seja, exatamente o perfil do Atom. Além da escolha do algoritmo, a configuração dos hiperparâmetros tem impacto direto na qualidade do aprendizado, o horizonte de coleta, ou seja, quantos passos o agente executa antes de cada atualização, precisa ser grande o suficiente para capturar comportamentos que se desenvolvem ao longo de vários passos, como a transferência de peso de uma perna para a outra, mas sem atrasar tanto as atualizações que o agente fique preso em comportamentos ruins por muito tempo. Kober et al. [15] alertam que o maior desafio do RL em robótica não está no algoritmo em si, mas no design da função de recompensa, assunto que abordamos em detalhe na Seção III.

### C. Locomoção Humanoide e Futebol de Robôs

O DRL também se mostra viável em robótica com muitos graus de liberdade mesmo sem um modelo explícito do ambiente, como já foi demonstrado por Peters et al. [24]. Nesse trabalho, os autores mostraram que algoritmos de gradiente de política podem funcionar em sistemas físicos reais, o que abriu caminho para uma linha de pesquisa que hoje apresenta resultados relevantes em várias plataformas robóticas. Heess et al. [12] ampliaram essa ideia ao mostrar que comportamentos locomotores complexos podem surgir a partir de objetivos simples de treinamento. Em simulações suficientemente realistas, padrões coordenados de caminhada, ajustes de postura e até recuperação de perturbações aparecem automaticamente quando o agente é treinado apenas para avançar sem cair. Para o Atom, isso tem uma implicação prática importante, não é necessário definir manualmente como cada perna deve se mover ou como as fases de apoio se alternam. O próprio algoritmo pode descobrir padrões eficientes de movimento, reduzindo um dos principais gargalos dos controladores baseados em ZMP.

Em controladores ZMP, ações como levantar o pé, transferir peso ou coordenar os braços para manter o equilíbrio normalmente precisam ser especificadas e ajustadas manualmente. Já com DRL, como mostram Heess et al. [12], esses comportamentos podem surgir durante o treinamento de forma conjunta. Por exemplo, o agente pode aprender a usar os braços para compensar a inércia do tronco durante a caminhada sem que essa estratégia seja programada explicitamente. No caso do Atom, que possui 6 graus de liberdade nos braços além dos 12 nas pernas, isso é extremamente útil, pois permite explorar mais possibilidades de movimento em vez de manter os braços fixos durante a locomoção.

A transferência de políticas treinadas em simulação para robôs reais ainda é um desafio conhecido, mas existem estratégias para reduzir esse problema. Haarnoja et al. [11] e Hwangbo et al. [13] apresentaram resultados positivos de locomoção aprendida por DRL aplicada em plataformas físicas. Em particular, Hwangbo et al. investigaram diretamente o problema de *sim-to-real transfer*. Nesse estudo, os autores modelaram os atuadores usando redes neurais em vez de assumir motores ideais, o que melhorou significativamente a transferência da política aprendida para o robô real. Essa abordagem é relevante para o Atom porque os servomotores Dynamixel XM540 e XM430 possuem características de resposta que nem sempre são bem representadas por modelos simplificados, especialmente sob cargas maiores durante a fase de apoio simples. Em outra linha de trabalho, Peng et al. [23] incorporaram referências de movimento capturadas de humanos durante o treinamento, gerando padrões de caminhada mais naturais. Embora esse não seja o foco imediato deste projeto, essa ideia pode ser explorada em trabalhos futuros quando houver interesse também na naturalidade do movimento.

As variáveis de sensor usadas no vetor de observações do agente foram escolhidas com base em trabalhos que analisaram empiricamente quais informações são importantes para manter o equilíbrio em robôs humanoides. Ha et al. [10] apresentam um survey recente sobre locomoção com DRL que discute quais tipos de sensores são mais relevantes nesse contexto. Shi et al. [28], por exemplo, mostraram que incluir sinais de contato dos pés e velocidade angular do tronco no vetor de observação melhora a estabilidade da política aprendida. Já Melo et al. [20] e Gonçalves [8] investigaram recuperação de empurrões em humanoides e observaram que políticas treinadas com DRL lidam melhor com colisões e perturbações externas do que controladores baseados em ZMP. Esses resultados ajudam a justificar a escolha das variáveis usadas no vetor de observações descrito na Seção III e indicam que a abordagem é adequada para o ambiente competitivo da RoboCup.

Outro ponto importante em DRL é a reprodutibilidade dos resultados. Duan et al. **duan2016** ob-

servaram que pequenas diferenças na configuração do treinamento podem levar a resultados bastante diferentes, mesmo quando o algoritmo e a tarefa são os mesmos. Parâmetros como taxa de aprendizado, tamanho do minibatch ou limites do espaço de ações podem influenciar significativamente o desempenho final da política. Isso cria um problema prático ao comparar resultados entre trabalhos diferentes, já que configurações ligeiramente distintas podem tornar as comparações injustas. No contexto deste projeto, isso significa que a avaliação do Atom deve controlar essas variáveis com cuidado. Por esse motivo, o desempenho final será analisado em partidas completas contra os mesmos adversários usados como referência na literatura, o que tende a produzir comparações mais confiáveis do que apenas observar curvas de recompensa em condições de treinamento diferentes.

No contexto específico do futebol de robôs, os trabalhos de Abreu et al. [1], [2] são extremamente relevantes para este projeto. O primeiro estudo mostra que é possível aprender a caminhar e chutar diretamente por meio de DRL, sem movimentos programados manualmente, utilizando a simulação da RoboCup 3D. Já o segundo apresenta políticas que alcançam desempenho comparável ao de equipes que desenvolveram seus controladores manualmente ao longo de vários anos. Para a RoboFEI, esses resultados sugerem que o investimento em DRL pode trazer ganhos práticos em competições. Maximo e Ribeiro [19] descrevem abordagens baseadas em ZMP utilizadas por equipes brasileiras, incluindo a própria RoboFEI, e apontam que esse tipo de controlador costuma exigir recalibração frequente e apresenta limitações quando há perturbações externas. Isso reforça a motivação para investigar o DRL como alternativa.

Um trabalho de referência direta para este projeto é o de Gianzanti [7], ex-aluno de mestrado do Centro Universitário FEI cujo estudo, ainda que não concluído, investigou o uso de DRL com o algoritmo PPO para controle de locomoção do robô Darwin OP3 da própria equipe RoboFEI. O trabalho resultou em dois repositórios públicos no GitHub, o *op3\_model*, que implementa um ambiente compatível com o OpenAI Gym [4] integrando o modelo do Darwin OP3 ao simulador MuJoCo [31], e o *op3\_trainer*, que contém o notebook de treinamento com o PPO. Esses repositórios têm valor prático direto para o presente trabalho porque documentam decisões de implementação concretas, como a definição do vetor de observações, o formato do espaço de ações e a estrutura da função de recompensa, tomadas especificamente para um robô humanoide da RoboFEI em MuJoCo. Ainda que o Darwin OP3 seja diferente do Atom, a experiência acumulada nesse estudo anterior serve como ponto de partida mais informado do que partir exclusivamente da literatura geral, e as lições aprendidas sobre instabilidades de treinamento e sensibilidade dos hiperparâmetros serão consideradas na configuração dos experimentos com o



Atom. Neste trabalho, o simulador escolhido foi o MuJoCo [31], principalmente por sua ampla utilização em pesquisas de locomoção e pela boa fidelidade na simulação de contatos entre corpos rígidos. Além disso, ele é compatível com o OpenAI Gym [4], o que facilita a integração com o código já desenvolvido pela BahiaRT dentro do cronograma disponível.

### III. Metodologia

A metodologia foi dividida em três etapas com dependências claras entre si. A primeira foi o levantamento bibliográfico, apresentado na seção anterior. A segunda é a modelagem do ambiente de simulação com o robô Atom, que atualmente está em estágio avançado. A terceira é o treinamento e a avaliação do agente, que está sendo conduzida em paralelo à segunda etapa à medida que partes do modelo ficam disponíveis para testes.

#### A. Modelagem do Ambiente de Simulação

A escolha do simulador foi uma das primeiras decisões técnicas do projeto e influencia diretamente todo o restante do processo. O MuJoCo [31] se destacou principalmente pelo uso de um integrador numérico especializado para corpos rígidos com restrições de contato. Na prática, isso permite simular a dinâmica de um robô humanoide de forma relativamente estável sem exigir hardware muito potente. Essa estabilidade é importante porque, em simuladores menos robustos, podem surgir inconsistências nas forças de contato quando o robô apoia o pé no chão ou sofre uma queda. Nesses casos, o agente pode acabar explorando artefatos numéricos da simulação em vez de aprender um padrão real de locomoção [5]. O PyBullet chegou a ser considerado como alternativa por ser gratuito e mais simples de instalar, mas apresenta menor fidelidade justamente na simulação de contatos. Como esse é um aspecto crítico para locomoção bípede, o uso dessa ferramenta acabou sendo descartado neste projeto, reforçando a escolha do MuJoCo.

Usar o simulador oficial da RoboCup 3D como ambiente de treinamento também foi uma decisão natural, em grande parte pelo conhecimento prévio das integrantes da RoboFEI com essa infraestrutura. Uma vantagem clara dessa escolha é que o agente aprende diretamente no ambiente onde irá competir. Isso inclui o atrito característico da grama sintética, as dimensões reais do campo e os estados de jogo típicos de uma partida. Dessa forma, reduz-se o risco de que uma política treinada em um ambiente simplificado apresente comportamento inesperado no simulador oficial durante a competição.

O ambiente utilizado é o *rcssservermj*, que é o servidor oficial da RoboCup 3D baseado em MuJoCo. Ele funciona em uma arquitetura cliente-servidor. O servidor é iniciado com a configuração do campo Adult Size, enquanto os agentes se conectam a ele por meio de soquetes TCP, informando parâmetros como endereço,

porta, número do jogador, nome do time e formato do campo. Esse tipo de arquitetura facilita a execução de vários agentes simultaneamente. No caso deste projeto, isso permitiu paralelizar o treinamento com oito ambientes ao mesmo tempo sem qualquer modificação no servidor, o que foi essencial para viabilizar o volume de treinamento necessário dentro do prazo disponível.

A arquitetura cliente-servidor também ajuda a separar as etapas de treinamento e avaliação, algo que nem sempre é feito em projetos de DRL. Durante o treinamento, os agentes utilizam uma política em modo de exploração, recebendo perturbações aleatórias nas ações para incentivar a exploração de novas regiões do espaço de estados. Isso permite observar o comportamento aprendido e compará-lo diretamente com o de equipes adversárias sem alterar a infraestrutura utilizada. Essa separação é importante porque avaliar a política ainda em modo de exploração pode levar a interpretações equivocadas sobre o desempenho do agente. Em muitos casos, a variabilidade observada não é sinal de aprendizado ruim, mas apenas resultado do ruído introduzido para fins de exploração. Adotar essa separação desde o início do projeto ajuda a evitar confusões e retrabalho durante fases mais avançadas do treinamento.

Como validação inicial da infraestrutura, o ambiente foi testado utilizando o modelo Booster T1 da BahiaRT [29]. A ideia foi garantir que eventuais problemas pudessem ser atribuídos ao ambiente ou à configuração do treinamento, e não ao modelo do robô em si. O modelo Booster T1 foi escolhido por ser o robô principal da equipe BahiaRT na categoria Adult Size da RoboCup 3D Simulation League, sendo amplamente testado e validado ao longo de diversas competições, o que o torna uma referência confiável para verificar se a infraestrutura funciona corretamente com diferentes geometrias e dinâmicas. O objetivo foi verificar se a infraestrutura funcionava corretamente e todos os experimentos executados nesses modelos ocorreram de forma estável e sem instabilidades numéricas perceptíveis. Com essa etapa concluída, foi possível ter confiança suficiente na infraestrutura para avançar para a fase seguinte do projeto, que envolve a modelagem do próprio robô Atom.

#### Modelagem do Atom no Onshape

Modelar o Atom no MuJoCo exigiu uma gama de ferramentas específica para garantir que o modelo físico fosse preciso o suficiente para o treinamento. O ponto de partida foram as peças modeladas pela equipe de mecânica do RoboFEI no Autodesk Fusion, que foram exportadas e importadas para o Onshape, plataforma CAD baseada em nuvem. A migração foi motivada pela disponibilidade da ferramenta *Onshape to Robot*, desenvolvida pela equipe Rhoban [3], [9], que automatiza a exportação do modelo para o formato MJCF nativo do MuJoCo, pois construir esse arquivo manualmente para um modelo com 20 juntas seria longo e

sujeito a erros difíceis de detectar sem análise cuidadosa da dinâmica. No Onshape, foram realizadas a montagem completa das peças, a atribuição de materiais e a definição das juntas cinemáticas, considerando que o Atom possui 20 juntas, conforme apresentado na Tabela 1.

Table 1: Distribuição das 20 juntas do robô Atom

| Região         | Juntas    | Movimentos principais      |
|----------------|-----------|----------------------------|
| Perna esquerda | 6         | Quadril, joelho, tornozelo |
| Perna direita  | 6         | Quadril, joelho, tornozelo |
| Braço esquerdo | 3         | Ombro, cotovelo            |
| Braço direito  | 3         | Ombro, cotovelo            |
| Cabeça         | 2         | Pan, tilt                  |
| <b>Total</b>   | <b>20</b> | —                          |

Os limites mecânicos das articulações do quadril e do tornozelo definem até onde cada junta pode girar no simulador sem gerar movimentos fisicamente impossíveis, e valores incorretos fariam a política aprendida adotar posturas que o hardware real não consegue replicar.

A definição das geometrias de colisão foi outro desafio importante na fase de modelagem, pois determina diretamente a fidelidade da dinâmica de contato simulada. O MuJoCo não usa a geometria visual das peças para calcular colisões, cada corpo rígido precisa de uma geometria simplificada, tipicamente caixas, cilindros ou cápsulas, que aproxime a forma real com custo computacional reduzido [13]. Uma geometria muito grosseira pode fazer o robô deslizar sobre o solo de forma não realista, enquanto uma muito detalhada aumenta o tempo de simulação a ponto de tornar o treinamento inviável dentro do prazo. O equilíbrio adotado priorizou a fidelidade nas regiões de contato mais frequentes durante a caminhada, especialmente solas dos pés e joelhos, e foi validado visualmente antes de qualquer treinamento, confirmando que as solas fazem contato correto com o solo.

A atribuição de materiais também é crítica porque o MuJoCo usa as propriedades físicas de cada peça para calcular a dinâmica do sistema, e densidades incorretas deslocariam o centro de massa para uma posição diferente da real [13]. Para as peças fabricadas por impressão 3D, foram usados materiais da biblioteca do Onshape com base nas especificações de fabricação. Para componentes eletrônicos sem correspondente direto na biblioteca, a densidade foi medida fisicamente e o material com valor mais próximo foi selecionado. A Tabela 2 resume os materiais e densidades atribuídos a cada grupo de componentes, qualquer erro nesses valores poderia fazer o agente aprender a compensar desequilíbrios que simplesmente não existem no

robô real, tornando a política inútil ao ser testada no hardware.

Table 2: Materiais atribuídos às peças do Atom no Onshape

| Componente                          | Material     | Dens. (lb/in <sup>3</sup> ) |
|-------------------------------------|--------------|-----------------------------|
| Tampa peito, porta bateria, puxador | PLA          | 0,045                       |
| Capacete, amortecedores             | ABS          | 0,038                       |
| NUC (computador)                    | Acrílico     | 0,042                       |
| Bateria                             | Teflon       | 0,078                       |
| Motores XM540/XM430                 | Fluoroelast. | 0,065                       |
| Peças estruturais                   | Al. 6061     | 0,098                       |

Três componentes possuem grande influência na dinâmica do sistema e merecem atenção especial na atribuição de densidades. A bateria, representada pelo Teflon (0,078 lb/in<sup>3</sup>), possui densidade consideravelmente maior que as peças plásticas e está posicionada no tronco, afetando diretamente a altura do centro de massa. Os motores Dynamixel, representados pelo Fluoroelastômero (0,065 lb/in<sup>3</sup>), estão distribuídos por todas as articulações e somam uma parcela relevante da massa total do robô. O computador NUC, representado pelo Acrílico (0,042 lb/in<sup>3</sup>), concentra massa significativa na região central do tronco e influencia diretamente a inércia rotacional do robô em torno dos eixos sagital e frontal, afetando a dinâmica de equilíbrio durante as fases de apoio simples. Qualquer erro na densidade de um desses componentes poderia levar o agente a compensar desequilíbrios que não existem no robô real, tornando a política inutilizável ao ser testada no hardware.

Com a montagem, os materiais e as juntas definidos, a ferramenta *Onshape to Robot* foi usada para exportar o modelo em formato MJCF, arquivo que descreve toda a estrutura do robô para o MuJoCo, incluindo geometria, massas, momentos de inércia, limites das juntas e relações cinemáticas entre os corpos. A Figura 1 apresenta o estado atual da montagem do robô no Onshape.

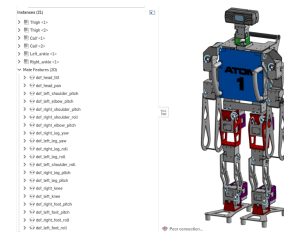


Figure 1: Montagem do robô Atom no Onshape com peças, materiais e 20 juntas definidos, pronto para exportação MJCF.

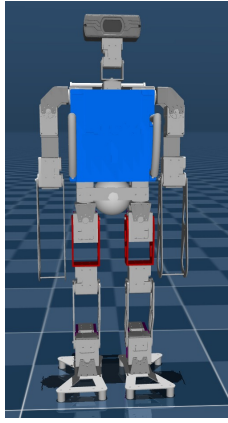


Figure 2: Visualização do centro de massa do robô Atom no Onshape, calculado a partir das densidades atribuídas a cada componente.

### Espaços de Observações e Ações

O vetor de observações do agente foi definido para oferecer ao controlador informação suficiente para saber a postura atual e a tendência de movimento do robô, sem variáveis redundantes. Com base em Haarnoja et al. [11], Hwangbo et al. [13], Ha et al. [10] e Shi et al. [28], o vetor inclui ângulos e velocidades das 20 articulações, velocidade linear e angular do centro de massa, orientação do tronco e sinais de contato dos pés com o solo, conforme a Tabela 3. Variáveis redundantes aumentariam desnecessariamente a dimensão do espaço de estado e prejudicariam a convergência, razão pela qual cada variável incluída tem um papel funcional específico no controle, conforme detalhado na terceira coluna da tabela.

Table 3: Vetor de observações planejado para o agente

| Variável                     | Dim.      | Papel no controle     |
|------------------------------|-----------|-----------------------|
| Ângulos das articulações     | 20        | Postura atual         |
| Velocidades das articulações | 20        | Dinâmica da postura   |
| Velocidade linear do CoM     | 3         | Estado global         |
| Velocidade angular do CoM    | 3         | Rotação do tronco     |
| Orientação (quaternion)      | 4         | Inclinação/equilíbrio |
| Contato pé esquerdo          | 1         | Apoio no solo         |
| Contato pé direito           | 1         | Apoio no solo         |
| <b>Total</b>                 | <b>52</b> | –                     |

O espaço de ações é composto pelos torques aplicados nas 20 articulações, com valores normalizados para  $[-1, 1]$  e reescalados para os limites físicos de cada junta. Essa normalização é necessária porque redes

neurais têm dificuldade em lidar com saídas em escalas muito diferentes entre articulações, sem ela os gradientes ficam desbalanceados e a convergência é bastante prejudicada [17]. A Figura 3 resume o processo completo desde a modelagem no Onshape até a avaliação final em partidas da RoboCup 3D, mostrando as dependências entre as etapas e deixando claro como cada decisão de modelagem impacta o treinamento.

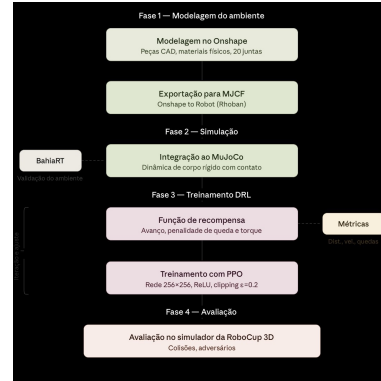


Figure 3: Fluxograma do processo, modelagem no Onshape, exportação MJCF, treinamento com PPO e avaliação em partidas da RoboCup 3D.

### B. Treinamento e Avaliação do Agente

#### Arquitetura da Rede Neural

A rede neural que representa a política do agente tem papel central no DRL e merece atenção especial antes de descrever os demais aspectos do treinamento. Em termos gerais, uma rede neural artificial é um modelo computacional inspirado na organização do cérebro, composto por camadas de unidades chamadas neurônios que transformam progressivamente a entrada em uma saída útil. No contexto do PPO aplicado à locomoção, a rede recebe o vetor de observações do robô como entrada, processa essa informação em camadas intermediárias e produz como saída a distribuição de probabilidade sobre as ações, que no caso do Atom corresponde aos torques a aplicar em cada articulação. A Figura 4 ilustra a arquitetura utilizada.

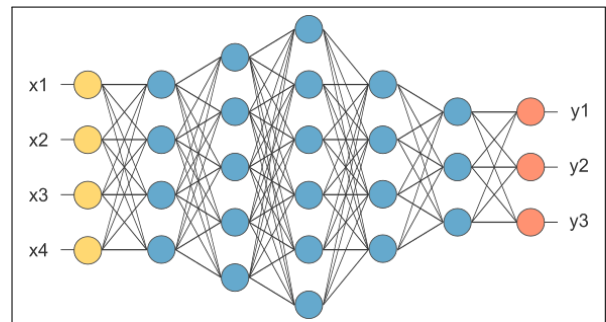


Figure 4: Arquitetura da rede neural utilizada pelo agente, com camada de entrada correspondente ao vetor de observações de dimensão 52, duas camadas ocultas densas de 256 neurônios com ativação ReLU e camada de saída com as ações contínuas normalizadas para as 20 articulações. Fonte: [22].

A arquitetura foi definida para equilibrar capacidade representacional e eficiência computacional dentro do hardware disponível. A rede é composta por duas camadas densas com 256 neurônios cada e ativação ReLU, configuração amplamente usada em estudos de locomoção bípede com DRL [duan2016](#), [11]. A função ReLU é definida como  $f(x) = \max(0, x)$  e é preferida por ser computacionalmente barata e por não sofrer do problema de gradiente desaparecente de forma tão acentuada quanto funções como a sigmoide, o que torna o treinamento mais estável em redes com múltiplas camadas [16]. Cada neurônio em uma camada calcula uma combinação linear ponderada das ativações da camada anterior, aplica a função de ativação e repassa o resultado para a camada seguinte, de forma que a rede vai extraindo representações progressivamente mais abstratas do estado do robô à medida que a informação avança pelas camadas. Essa capacidade de construir representações hierárquicas é exatamente o que permite ao DRL lidar com um vetor de entrada de 52 dimensões sem que o engenheiro precise definir manualmente quais combinações de sensores são relevantes para o equilíbrio. Com menos neurônios, o modelo pode não ter capacidade suficiente para representar a política de locomoção, com mais, a convergência fica mais lenta sem ganho proporcional de desempenho. O servidor da equipe tem uma GPU com memória limitada, e a configuração adotada foi verificada em testes de carga antes do treinamento principal, confirmando que os oito ambientes paralelos podem ser executados sem saturar a memória disponível.

O trabalho anterior de Giazanti [7] com o Darwin OP3 no mesmo simulador MuJoCo utilizou uma arquitetura de rede com estrutura semelhante, e os resultados parciais documentados nos repositórios indicam que essa configuração é suficiente para representar políticas de locomoção em robôs humanoides de porte similar ao Atom. Essa observação reforça a escolha feita neste projeto e reduz o risco de que a arquitetura seja um gargalo no treinamento, permitindo concentrar o esforço de ajuste na função de recompensa e nos hiperparâmetros do PPO.

O embaralhamento das experiências coletadas nos ambientes paralelos antes de cada atualização é um detalhe técnico com impacto direto na estabilidade do treinamento. No PPO, as experiências de todos os ambientes são concatenadas em um único buffer e embaralhadas antes de serem divididas em minibatches para as atualizações de gradiente, o que garante que cada minibatch contenha amostras de diferentes ambientes e de diferentes fases da trajetória, reduzindo a correlação entre amostras consecutivas que poderia desestabilizar o gradiente estimado. Para o Atom, essa propriedade é útil porque a dinâmica do robô nos oito ambientes paralelos pode divergir rapidamente após poucas etapas, gerando amostras com distribuições distintas de estados. Combinadas após o embaralhamento,

essas amostras cobrem melhor o espaço de situações que o agente precisa aprender a lidar, acelerando a convergência para políticas mais robustas.

Os hiperparâmetros do PPO serão definidos com base nos valores padrão reportados em Schulman et al. [27] e ajustados a partir das observações dos treinamentos práticos. A configuração inicial herdará os valores já calibrados pela BahiaRT para o simulador da RoboCup 3D, o que reduz o número de decisões arbitrárias no início do treinamento e fornece um ponto de partida mais informado do que simplesmente usar os defaults da biblioteca. Aqui está o trecho reescrito para deixar claro que o Atom será inserido no código do Giazanti antes de integrá-lo ao ambiente da RoboCup 3D:

Os experimentos de Giazanti [giazanti2022](#) com o Darwin OP3 também servirão como referência para a definição do horizonte de coleta e do tamanho dos minibatches, pois esses parâmetros foram explorados no contexto de um robô humanoide da mesma equipe no mesmo simulador. A estratégia adotada será inserir o modelo MJCF do Atom diretamente no código de Giazanti [7], substituindo o Darwin OP3, de forma a reaproveitar a infraestrutura de treinamento já validada antes de migrar o agente para o ambiente oficial da RoboCup 3D. Essa abordagem permite isolar eventuais problemas de integração do modelo em um ambiente mais controlado, reduzindo o risco de diagnósticos equivocados durante o treinamento principal. Esses valores serão ajustados conforme os primeiros experimentos com o Atom revelarem a necessidade.

## Função de Recompensa

A função de recompensa é o componente mais crítico do aprendizado por reforço em robótica. Kober et al. [15] destacam que escolhas inadequadas podem levar o agente a comportamentos indesejados, mesmo com algoritmos corretos. No caso deste projeto, a função foi definida para incentivar avanço e penalizar quedas e esforço excessivo. Isso garante que o agente não apenas se mova, mas também mantenha estabilidade e eficiência energética. Assim, a definição cuidadosa da recompensa é essencial para que o comportamento aprendido seja útil em cenários reais de jogo.

$$r_t = w_v \cdot v_x - w_f \cdot 1[\text{queda}] - w_e \cdot \|\tau_t\|^2 \quad (4)$$

onde  $v_x$  é a velocidade de avanço,  $1[\text{queda}]$  é um indicador binário de queda,  $\tau_t$  é o vetor de torques aplicados e  $w_v$ ,  $w_f$ ,  $w_e$  são pesos que equilibram os três objetivos. A penalidade quadrática sobre os torques induz o agente a desenvolver uma caminhada energeticamente mais eficiente, já que sem ela o agente pode aprender a usar torques elevados que funcionam em simulação, mas que aqueceriam ou danificariam



os servomotores Dynamixel em uso prolongado. A observação dos jogos com a BahiaRT deixou claro que a recuperação rápida após quedas é mais determinante para o desempenho competitivo do que a velocidade de avanço, o que orientará o ajuste de  $w_f$  para um valor suficientemente alto sem tornar o aprendizado tão conservador que o robô se recuse a avançar.

### Métricas de Avaliação

Quatro métricas principais serão registradas ao longo do treinamento para garantir que a política aprendida realmente evolui na direção correta. As métricas são distância percorrida por episódio, velocidade média de caminhada, altura do centro de massa e taxa de quedas [duan2016](#), [15]. A curva de recompensa total isolada não é suficiente para avaliar a qualidade da política porque um agente pode apresentar recompensa crescente enquanto aprende a permanecer parado em uma postura estável sem avançar, e as métricas de distância e velocidade identificariam esse comportamento imediatamente, permitindo corrigir a função de recompensa antes de desperdiçar horas de treinamento. Avaliações visuais periódicas no simulador também serão conduzidas, pois aspectos como a naturalidade da passada e a qualidade da recuperação de tropeços não são completamente capturados por métricas numéricas [23], e a avaliação final ocorrerá em partidas simuladas completas da RoboCup 3D, cenário que MacAlpine e Stone [18] e Abreu et al. [1] identificam como o mais adequado para distinguir políticas de locomoção de boa qualidade.

## IV. Resultados Preliminares

Os resultados obtidos até o momento cobrem duas frentes, a validação do ambiente de simulação com o modelo da BahiaRT e o início da modelagem do Atom. Nenhum treinamento com o Atom foi executado ainda, pois o modelo não está completamente integrado ao MuJoCo. Ainda assim, cada etapa concluída reduziu o risco das que virão, porque os problemas foram identificados e resolvidos em um contexto controlado em vez de aparecerem durante um treinamento longo e de difícil diagnóstico.

### A. Validação do Ambiente

A validação do ambiente com o modelo Booster T1 foi a primeira etapa prática do projeto e serviu para isolar potenciais problemas de infraestrutura antes de introduzir a complexidade do Atom. O grupo optou por usar o Booster T1 da BahiaRT [29] porque qualquer problema encontrado poderia ser claramente atribuído ao ambiente ou à configuração do treinamento, e não ao modelo do robô em si, além da familiaridade das integrantes com o time BahiaRT por conta das competições de robótica. O Booster T1 é o robô principal da equipe BahiaRT na categoria Adult Size, tendo sido amplamente testado em competições internacionais, o que garante que seu comportamento

em simulação serve como referência confiável para detectar anomalias na infraestrutura. Essa estabilidade confirmada foi o sinal de que a infraestrutura estava pronta para receber o Atom.

Simular uma partida completa no formato Adult Size com os robôs Booster T1 revelou um dado importante sobre a dinâmica do jogo que influenciará diretamente a calibração da função de recompensa. A simulação nos permitiu observar os diferentes estados de jogo como partida, bola em campo, falta, queda e reinício, deixando bem evidente que a recuperação rápida após quedas é muito mais determinante para o desempenho competitivo do que a velocidade de caminhada em campo aberto. Um robô que cai e demora para se levantar perde posicionamento, deixa o adversário livre e frequentemente concede gols. Essa observação vai influenciar diretamente a calibração do peso  $w_f$  na função de recompensa, que precisará ser alto o suficiente para que o agente aprenda a priorizar o equilíbrio acima de tudo.



Figure 5: Partida simulada no formato Adult Size com os robôs Booster da equipe BahiaRT no simulador oficial da RoboCup 3D, utilizada para validação do ambiente.

### B. Modelagem do Atom

A modelagem do Atom avançou de forma consistente, embora com algumas dificuldades maiores do que o previsto, principalmente na etapa de determinar os pesos e as densidades de cada peça. Como essas informações nem sempre estavam disponíveis de forma clara na documentação do robô, foi necessário estimar ou pesquisar os materiais utilizados e ajustar manualmente as densidades para que a massa total ficasse próxima da esperada. Esse processo exigiu revisar várias peças individualmente e fazer pequenos ajustes até obter valores coerentes para a simulação. Apesar de ter sido resolvido de forma iterativa, essa etapa acabou consumindo mais tempo do que o planejado e reforça a importância de registrar desde o início os parâmetros físicos do hardware, como massa e material das peças, em projetos que envolvem modelagem física detalhada.

Partes do modelo já foram exportadas para MJCF e carregadas no simulador para testes de visualização, confirmando que a estrutura cinemática está correta e que o modelo pode ser instanciado sem erros. A próxima etapa é completar a integração e validar o comportamento físico do modelo, verificando posição do centro de massa, resposta das juntas a torques e

estabilidade do contato com o solo, antes de iniciar qualquer treinamento. Um resultado concreto desta fase foi entender em detalhe o processo de inserção de um novo modelo no simulador da RoboCup 3D, que não estava documentado de forma acessível e exigiu estudo cuidadoso do código-fonte, sua documentação vai constituir um recurso valioso para equipes futuras que trabalharão com a mesma plataforma.

## V. Discussão

Os resultados preliminares indicam que o projeto está avançando de forma consistente com o planejamento inicial, e as decisões metodológicas adotadas até o momento se mostraram adequadas para o desenvolvimento do trabalho. A realização da validação utilizando o modelo da BahiaRT em paralelo com a modelagem do Atom foi uma decisão importante para manter o progresso do projeto. Ao trabalhar simultaneamente com um modelo já existente e com o novo modelo em desenvolvimento, o grupo conseguiu testar conceitos, verificar comportamentos do sistema e compreender melhor como integrar os diferentes componentes da simulação antes que o Atom estivesse completamente finalizado. Essa abordagem reduziu incertezas no processo, pois permitiu identificar possíveis problemas de implementação e entendimento da arquitetura do sistema de forma antecipada.

Além disso, a utilização do modelo da BahiaRT como referência prática facilitou o aprendizado técnico da equipe durante o desenvolvimento do projeto. A análise do funcionamento desse modelo permitiu observar como diferentes partes do sistema interagem, como os dados de percepção são processados e como o controle de locomoção pode ser estruturado dentro do ambiente de simulação. Isso contribuiu para que o grupo tomasse decisões mais informadas durante a construção do Atom, evitando retrabalho e ajudando a alinhar as escolhas técnicas com soluções que já demonstraram funcionar em contextos semelhantes.

Essa estratégia metodológica também contribuiu para o cumprimento do cronograma do projeto. Como diferentes frentes de trabalho puderam evoluir ao mesmo tempo, o grupo conseguiu manter um ritmo constante de desenvolvimento, mesmo enquanto algumas etapas da modelagem do Atom ainda estavam em andamento. Esse paralelismo nas atividades permitiu aproveitar melhor o tempo disponível, garantindo progresso contínuo e mantendo o projeto alinhado com os objetivos definidos para esta fase do trabalho.

A principal incerteza que ainda permanece diz respeito à qualidade do modelo físico do Atom para o treinamento, e ela precisa ser reconhecida honestamente como uma limitação do trabalho. Mesmo com todos os cuidados adotados, o modelo MJCF é uma aproximação, geometrias de colisão são simplificadas, atuadores são tratados como motores ideais e efeitos como folga mecânica e atrito nas juntas não são completamente capturados, o que caracteriza o *sim-to-real*

*gap* [13]. Para o escopo deste trabalho, o objetivo não é resolver esse *gap* completamente, mas produzir uma política competitiva em simulação como prova de conceito, deixando a transferência para hardware como trabalho futuro. Uma forma direta de verificar a qualidade do modelo antes do treinamento é comparar as frequências naturais de oscilação do robô físico com as do modelo simulado, e se diferirem significativamente, os parâmetros de massa ou inércia precisam ser revisados antes de iniciar o treinamento principal.

A *domain randomization* é a técnica mais promissora para reduzir o *sim-to-real gap* em trabalhos futuros com o Atom. Ela consiste em variar aleatoriamente parâmetros do modelo durante o treinamento, como massas, coeficientes de atrito e atrasos nos atuadores, tornando a política aprendida mais robusta a variações [13]. Peng et al. [23] mostram que combinar *domain randomization* com referências de movimento melhora ainda mais a qualidade das políticas aprendidas. Aplicar essas técnicas ao Atom está previsto como extensão futura, especialmente se os experimentos iniciais revelarem diferenças expressivas entre o desempenho em simulação e o comportamento esperado no hardware, e por isso a infraestrutura de treinamento está sendo construída de forma a facilitar essa extensão.

Uma questão que ainda precisa ser investigada no projeto é qual estratégia de aprendizado será mais adequada para o Atom. A definição dessa estratégia é importante porque as características dinâmicas do robô ainda não foram totalmente validadas no simulador. No código da BahiaRT, por exemplo, o agente começa o treinamento diretamente em condições completas de jogo. Essa abordagem funciona bem para o Booster T1 porque sua configuração física já foi testada e refinada ao longo de diversas competições, o que reduz a chance de comportamentos inesperados durante o treinamento. Para o Atom, porém, pode ser mais eficiente começar com um processo de aprendizado mais gradual. Uma alternativa é utilizar um currículo progressivo [12], no qual o agente começa resolvendo tarefas simples e a dificuldade aumenta aos poucos. No início, o objetivo poderia ser apenas manter o robô em pé por alguns segundos ou realizar pequenos movimentos estáveis. Com o tempo, novos desafios seriam adicionados até que o agente consiga lidar com as condições completas do ambiente de jogo. Essa abordagem já apresentou bons resultados em outros trabalhos envolvendo locomoção humanoide. Estudos como [20] e [8] mostram que começar com tarefas mais simples pode ajudar o agente a aprender comportamentos básicos antes de lidar com situações mais complexas.

Encontrar bons valores para os pesos da função de recompensa é outro aspecto principal que precisará de atenção dedicada nas primeiras semanas de treinamento. Valores muito altos para  $w_f$  levam o agente a permanecer parado para evitar quedas, enquanto val-

ores muito baixos para  $w_e$  resultam em uma caminhada funcionalmente correta, mas com torques que danificariam os servomotores em uso prolongado. Como alertam Kober et al. [15], esse ajuste é inevitável em RL para robótica e precisa ser tratado como parte central da metodologia, não como detalhe de implementação.

Em relação às limitações computacionais, o treinamento do agente exige um volume considerável de simulação para alcançar bons resultados. Pelas estimativas iniciais, serão necessárias cerca de 2 a 4 semanas de treinamento contínuo para atingir uma convergência satisfatória, e esse período já foi considerado no cronograma do projeto. Para tornar esse processo viável dentro do tempo disponível, a paralelização do treinamento é fundamental. Utilizando 8 ambientes de simulação rodando ao mesmo tempo, o agente consegue coletar experiências muito mais rapidamente, o que acelera o aprendizado. Além disso, a arquitetura cliente-servidor do *rcssservermj* permite executar esses ambientes em paralelo sem necessidade de modificar o servidor. A transferência da política para o robô físico está fora do escopo deste trabalho. O projeto se concentra na validação em simulação, que já representa um avanço importante por permitir estudar e desenvolver comportamentos de locomoção de forma controlada e com menor custo experimental.

Do ponto de vista das contribuições para a RoboFEI, o projeto também gera resultados práticos que continuam úteis independentemente dos resultados numéricos do treinamento. Um deles é o modelo MJCF calibrado do Atom, que exigiu um trabalho extenso de medição, montagem e ajustes para representar corretamente o robô no simulador. Esse modelo pode servir como base para futuros projetos que envolvam simulação do Atom. Outro resultado importante é a documentação do processo de inserção de novos modelos no servidor oficial da RoboCup 3D. Como esse processo não estava disponível de forma clara, foi necessário reconstruí-lo a partir da análise do código-fonte, e essa documentação pode facilitar o trabalho de futuros integrantes da equipe.

Além desses resultados técnicos, o projeto também abre caminho para uma nova frente de atuação da equipe, o futebol de robôs em simulação 3D. Ao desenvolver o modelo do robô, a infraestrutura de treinamento e a integração com o ambiente de simulação, o trabalho cria a base necessária para que a RoboFEI possa explorar essa categoria no futuro. Isso amplia as possibilidades de pesquisa da equipe e permite testar estratégias, algoritmos de aprendizado e comportamentos complexos sem depender apenas de experimentos com o robô físico. Dessa forma, o projeto não contribui apenas com resultados pontuais, mas também estabelece uma base técnica que pode viabilizar a participação da equipe na categoria de futebol de robôs em simulação 3D nos próximos anos.

## VI. Cronograma

A Tabela 4 apresenta o cronograma previsto para as etapas restantes até o fim do primeiro semestre de 2026. As atividades foram organizadas respeitando as dependências entre si, e a sobreposição entre a redação da monografia e as demais atividades é intencional, pois permite que os resultados sejam documentados à medida que surgem, evitando a concentração de trabalho de escrita no final do período.

Table 4: Cronograma para o 1º semestre de 2026

| Atividade                | Jan | Fev | Mar | Abr | Mai | Jun |
|--------------------------|-----|-----|-----|-----|-----|-----|
| 1. Início das pesquisas  | •   |     |     |     |     |     |
| 2. Def. observações      | •   | •   |     |     |     |     |
| 3. Teste do simulador    |     |     | •   | •   |     |     |
| 4. Revisão bibliográfica |     | •   | •   | •   | •   | •   |
| 5. Entrega/apresent.     |     |     |     |     |     | •   |

## VII. Referências

- [1] M. Abreu, N. Lau e L. P. Reis, “Learning Low-Level Motor Skills for Soccer Playing Humanoid Robots,” em *IEEE ICARSC*, Santa Maria da Feira, Portugal, 2021, pp. 1–8.
- [2] M. Abreu, N. Lau, A. Sousa e L. Reis, “Learning Low-Level Skills from Scratch for Humanoid Robot Soccer Using Deep Reinforcement Learning,” em *IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, Porto, Portugal, 2019, pp. 1–8.
- [3] J. Allali et al., “Rhuban Football Club 2023 Champion Team Paper,” em *RoboCup 2023*, Cham: Springer, 2024.
- [4] G. Brockman et al. “OpenAI Gym,” acessado em 25 de mar. de 2026. URL: <https://arxiv.org/abs/1606.01540>.
- [5] E. Coumans e Y. Bai, *PyBullet: Physics Simulation for Robotics*, 2021. acessado em 2 de abr. de 2026. URL: <http://pybullet.org>.
- [6] S. Fujimoto, H. van Hoof e D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” em *International Conference on Machine Learning (ICML)*, Stockholm, Sweden, 2018, pp. 1587–1596.
- [7] F. Giansanti. “op3\_model e op3\_trainner.” Disponível também em: [https://github.com/Giansanti/op3\\_trainner](https://github.com/Giansanti/op3_trainner), acessado em 10 de abr. de 2026. URL: [https://github.com/Giansanti/op3\\_model](https://github.com/Giansanti/op3_model).

- [8] R. F. Gonçalves, “Deep Reinforcement Learning Application in Humanoid Robots Locomotion,” tese de mestrado, Universidade Estadual de Campinas, Campinas, SP, 2021.
- [9] L. Gondry et al., “Rhoban Football Club 2019 Champion Team Paper,” em *RoboCup 2019*, Cham: Springer, 2019, pp. 491–503.
- [10] S. Ha et al., “Learning-Based Legged Locomotion: State of the Art and Future Perspectives,” *The International Journal of Robotics Research*, vol. 44, n.º 1, pp. 1–43, 2025.
- [11] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine. “Learning to Walk via Deep Reinforcement Learning,” acessado em 12 de mar. de 2026. URL: <https://arxiv.org/abs/1812.11103>.
- [12] N. Heess et al. “Emergence of Locomotion Behaviours in Rich Environments,” acessado em 22 de mar. de 2026. URL: <https://arxiv.org/abs/1707.02286>.
- [13] J. Hwangbo et al., “Learning Agile and Dynamic Motor Skills for Legged Robots,” *Science Robotics*, vol. 4, n.º 26, eaau5872, 2019.
- [14] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda e E. Osawa, “RoboCup: The Robot World Cup Initiative,” em *International Conference on Autonomous Agents*, Marina del Rey, CA, 1997, pp. 340–347.
- [15] J. Kober, A. Bagnell e J. Peters, “Reinforcement Learning in Robotics: A Survey,” *The International Journal of Robotics Research*, vol. 32, n.º 11, pp. 1238–1274, 2013.
- [16] Y. LeCun, Y. Bengio e G. Hinton, “Deep Learning,” *Nature*, vol. 521, n.º 7553, pp. 436–444, 2015.
- [17] T. P. Lillicrap et al., “Continuous Control with Deep Reinforcement Learning,” em *International Conference on Learning Representations (ICLR)*, 2016.
- [18] P. MacAlpine e P. Stone, “UT Austin Villa: RoboCup 3D Simulation League Champions,” em *RoboCup 2017: Robot World Cup XXI*, Cham: Springer, 2018, pp. 473–485.
- [19] M. R. O. A. Maximo e C. H. C. Ribeiro, “Omni-directional Zero Moment Point Walking Biped Robot,” *Journal of Control, Automation and Electrical Systems*, vol. 30, n.º 4, pp. 519–531, 2019.
- [20] D. Melo, M. R. O. A. Maximo e A. Cunha, “Learning Push Recovery Behaviors for Humanoid Walking Using Deep Reinforcement Learning,” *Journal of Intelligent and Robotic Systems*, vol. 105, n.º 2, pp. 1–18, 2022.
- [21] V. Mnih et al., “Human-Level Control through Deep Reinforcement Learning,” *Nature*, vol. 518, n.º 7540, pp. 529–533, 2015.
- [22] U. F. de Ouro Preto. “Fundamentos de Redes Neurais,” acessado em 5 de abr. de 2026. URL: <https://www2.decom.ufop.br/imobilis/fundamentos-de-redes-neurais/>.
- [23] X. B. Peng et al., “AMP: Adversarial Motion Priors for Stylized Physics-Based Character Control,” em *ACM SIGGRAPH*, 2021.
- [24] J. Peters, S. Vijayakumar e S. Schaal, “Reinforcement Learning for Humanoid Robotics,” em *IEEE-RAS International Conference on Humanoid Robots*, Karlsruhe, Germany, 2003, pp. 1–20.
- [25] M. Riedmiller, “Neural Fitted Q Iteration,” em *European Conference on Machine Learning (ECML)*, Porto, Portugal, 2005, pp. 317–328.
- [26] J. Schulman et al., “Trust Region Policy Optimization,” em *International Conference on Machine Learning (ICML)*, Lille, France, 2015, pp. 1889–1897.
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov. “Proximal Policy Optimization Algorithms,” acessado em 18 de mar. de 2026. URL: <https://arxiv.org/abs/1707.06347>.
- [28] Q. Shi, Y. Zhang e Z. Li, “Deep Reinforcement Learning-Based Attitude Motion Control for Humanoid Robots with Stable Training Strategies,” *Industrial Robot: The International Journal of Robotics Research and Application*, vol. 47, n.º 5, pp. 741–752, 2020.
- [29] M. Simões et al., “BahiaRT 2022: Team Description Paper,” em *RoboCup 2022*, Bangkok, Thailand, 2022.
- [30] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2ª ed. Cambridge, MA: MIT Press, 2018.
- [31] E. Todorov, T. Erez e Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Vilamoura, Portugal, 2012, pp. 5026–5033.
- [32] M. Vukobratović e B. Borovac, “Zero-Moment Point — Thirty Five Years of Its Life,” *International Journal of Humanoid Robotics*, vol. 1, n.º 1, pp. 157–173, 2004.
- [33] C. J. C. H. Watkins e P. Dayan, “Q-Learning,” *Machine Learning*, vol. 8, n.º 3–4, pp. 279–292, 1992.